DATA STRUCTURES

ASSIGNMENT ANSWERS

NAME: BALAJI.S.A

REG NO: 1921260146

COURSE: DATA STRUCTURE

COURSE CODE: CSA0305

A. Define Primitive Data Structures with Suitable Examples

Definition: Primitive data structures are the basic data types provided by a programming language. They store simple values and are used to build more complex data structures.

Examples

Integer (int): Stores whole numbers.

```
int age = 18;
```

Float (float): Stores decimal numbers.

```
float marks = 85.5;
```

Character (char): Stores a single character.

```
char grade = 'A';
```

Boolean (bool): Stores true or false values in languages that support this type directly.

```
bool isPassed = true;
```

Double (double): Stores decimal numbers with greater precision than float in many languages.

```
double pi = 3.141592;
```

Conclusion: Primitive data types are the foundation of programming because they allow basic values to be stored and processed efficiently.

B. Analyze Nested Recursion with Recurrence Relations

Nested recursion occurs when a recursive function makes a recursive call using the result of another recursive call as an argument. In simple terms, one recursive call is placed inside another.

Example: A general nested-recursive function may look like this:

```
int F(int n) {
```

if (n <= 1) return 1;

return F(F(n - 1));

```
}
```

A recurrence relation expresses the running time of an algorithm in terms of smaller subproblems. A general form for a nested recursive function can be written as:

$T(n) = T(n – 1) + T(g(n)) + c$

Here, g(n) represents the input produced by the inner recursive call, and c is the constant work done outside the recursive calls. The exact recurrence depends on the function; nested recursion has no single universal time complexity.

Steps to analyze

Identify the base case where recursion stops.

Count the recursive calls made during each invocation.

Write a recurrence relation for the running time.

Expand or solve the recurrence to estimate time and space complexity.

Conclusion: Recurrence relations help estimate the time and space required by nested recursive algorithms.

C. Explain the Use of Arrays in Scientific Computing

An array is a collection of elements of the same data type stored in consecutive memory locations. Each element is accessed using an index. Arrays are useful in scientific computing because they efficiently store and process large sets of numerical data.

Main uses of arrays

Storing experimental data: Measurements such as temperature, pressure, and voltage can be stored in arrays.

Mathematical calculations: Arrays represent vectors, matrices, and numerical sequences used in algebra and numerical methods.

Scientific simulations: They store particle positions, fluid velocities, or temperatures across a grid.

Data analysis: Arrays help calculate sums, averages, standard deviations, and other statistical values.

Image and signal processing: Images can be represented as two-dimensional arrays of pixels, while digital signals can be stored as one-dimensional arrays.

Efficiency: Indexed access is fast, and numerical libraries can perform operations efficiently on large datasets.

Example in Python:

```
temperatures = [25.5, 26.0, 27.2, 26.7]
average = sum(temperatures) / len(temperatures)
print(average)
```

Conclusion: Arrays are essential in scientific computing because they organize numerical data and support efficient calculations, simulations, and analysis.